Today

Welcome to your next lesson: Many objects, one class.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and the best ways to help you get stronger.

Keep it up, every step counts.

Imagine you have two dogs, Luna and Max 🐶🐶

They start the day with the same happiness level: 5 out of 10.

Later, you pet only Luna once. Max gets no pets (poor Max).

At the end:

Luna’s happiness: 6
Max’s happiness: 5

Same type of pet, same starting state, but each dog keeps their own happiness number.

In Python, we model this by:

One class (like a blueprint: “Dog”)
Two objects (Luna and Max), each with its own happiness

We’ll practice creating several objects from one class and then changing just one of them.

Before we touch code:

Does the Luna/Max story feel clear: same starting happiness, but only Luna’s number should change when you pet her?

Yes, that makes sense

Nice 👍

Let’s mirror that story in Python with a simple pet class.

Here’s the idea step by step:

We make one class Pet with an __init__ that sets happiness = 5 for every new pet.
We create two objects from it: luna and max.
At the start, both have the same state.

Visually:

Before any methods:

luna → happiness: 5
max → happiness: 5

Next step will be: call a method on only one of them and see what happens.

Make sure this picture is solid in your head:

If the code does:

Python
luna_happiness = 5
max_happiness = 5
Do you agree these are two separate variables that just happen to both be 5?

Yes, they’re separate

Perfect 👌 That’s exactly how to think about objects too.

Now let’s turn this into a real Python example with a class and two pets.

Step by step, we’ll:

Create a Pet class.
Give it:
an __init__ that sets happiness to 5
a pet() method that adds 1 to happiness
Create luna and max from that class.
Call pet() only on luna, then print both happiness values.
Here’s what I’d like you to do now:

In the code editor, write Python code that:

Defines this class:

```
class Pet:
    def __init__(self):
        self.happiness = 5

    def pet(self):
        self.happiness = self.happiness + 1
```

Then, below the class:

```
luna = Pet()
max = Pet()

luna.pet()

print("Luna happiness:", luna.happiness)
print("Max happiness:", max.happiness)
```

Run it, then Submit code so I can see it.

Don’t tell me the output yet—we’ll reason about it together after I see your code.

Python

```
class Pet:
    def __init__(self):
        self.happiness = 5

    def pet(self):
        self.happiness = self.happiness + 1

luna = Pet()
max = Pet()
luna.pet()
```

```
print("Luna happiness:", luna.happiness)
print("Max happiness:", max.happiness)
```
Nice work, Ron — your code is exactly what I was aiming for ✅

You just showed two key things:

Both objects start with the same state:

luna.happiness → 5
max.happiness → 5
After calling luna.pet():

luna.happiness → 6
max.happiness → still 5

So changing one object's state did not affect the other. Each object keeps its own copy of happiness.

Quick check-in 🧠

In the line luna.pet(), self inside the pet method refers to Luna, not Max.

Does that connection feel clear: luna.pet() means "run pet with self = luna"?

Yes, that's clear

What do you think? Pick the answer that matches your understanding 👇

Luna and Max both start with happiness 5. After running luna.pet() once, what are their happiness values?

A.
Luna: 5, Max: 5

B.
Luna: 6, Max: 5

C.
Luna: 6, Max: 6

D.
Luna: 5, Max: 6

Exactly right 🎯

After luna.pet() runs once:

Luna: happiness goes from 5 to 6
Max: stays at 5

Reason step by step:

luna = Pet() → luna.happiness = 5
max = Pet() → max.happiness = 5 (separate object)
luna.pet() → inside pet, self is Luna, so only luna.happiness becomes 6. Nothing touches max.

So you've nailed the big idea: same class, identical start, but each object's state changes independently 💪

Want to do one more small challenge where we call pet() a different number of times on each pet and compare their happiness?

Give me the challenge

Love it 🔥 Let's step it up just a bit.

Here's the challenge, step by step:

Keep the same Pet class you already wrote.

In the code editor, below the class:

Create two pets: luna = Pet() and max = Pet()
Call pet():

luna.pet() three times

```
max.pet() once
Then print both happiness values, like:

print("Luna happiness:", luna.happiness)
print("Max happiness:", max.happiness)

Don’t tell me the result in chat yet—just Run and then Submit code so I can see what happened.

Python
class Pet:
    def __init__(self):
        self.happiness = 5

    def pet(self):
        self.happiness = self.happiness + 1

luna = Pet()
max = Pet()
luna.pet()
luna.pet()
luna.pet()
max.pet()

print("Luna happiness:", luna.happiness)
print("Max happiness:", max.happiness)
Nice, Ron — this is spot on ✅


You showed:


Start: both at 5
luna.pet() three times → 5 → 6 → 7 → 8
max.pet() once → 5 → 6

So the final state is:


luna.happiness → 8
max.happiness → 6

Same class, same starting state, but their paths diverged based on how many times you called pet() on each. That’s exactly the idea of independent object state 🎯

Quick check before we wrap this lesson concept:
```

If we added a third pet rocky = Pet() and never called rocky.pet(), what would rocky.happiness be right after creation? Just answer with the number.

5

Exactly 💯 — rocky.happiness would be 5 right after creation.

So now you've shown you can:

Create multiple objects from one class with the same starting state
Change one object's state without affecting the others
Compare their states after different method calls

You've basically mastered everything in this lesson 🎓

Want to wrap up this lesson and move on to the next one, or do you want one more twisty example with a different kind of class (not pets) before we finish?

Move to the next lesson

Nice work, Ron 🎉

Quick recap of what you nailed in this lesson:

You created multiple objects from the same class (Pet) with the same starting state (happiness = 5).
You used pet() on some objects more times than others and saw that:
each object's happiness changed independently
calling a method on luna never affected max, and vice versa
You correctly reasoned about final states just by tracing the method calls.

This is core object-oriented thinking: one blueprint, many independent objects.

```
You’re all set for the next lesson 🚀

When you’re ready, hit Next Lesson in the Learn tab and we’ll
keep building on this.




class Pet:
    def __init__(self):
        self.happiness = 5

    def pet(self):
        self.happiness = self.happiness + 1

luna = Pet()
max = Pet()
luna.pet()
luna.pet()
luna.pet()
max.pet()

print("Luna happiness:", luna.happiness)
print("Max happiness:", max.happiness)
```